

Segmentacija parkirnih zona pomoću računalnog vida

(Deep Learning pristup – detekcija parkirnih mjesta)

Projektna dokumentacija

Josip Šimić i Josip Previšić

Sadržaj

1. Uvod	3
2. Računalno okruženje i odabir hardvera	4
3. Spajanje Google Drive-a	5
4. Skup podataka i podjela	5
6. Konfiguracijska datoteka data.yaml	11
7. Instalacija YOLOv8 biblioteke	12
8. Treniranje YOLOv8 modela	12
9. Evaluacija modela	14
9.1 Evaluacija na validation skupu	14
9.2 Konačna evaluacija na testnom skupu	14
9.3 Analiza konfuzijske matrice	15
10. Vizualna demonstracija (inferencija)	17
11. Spremanje rezultata	18
12. Ponovna upotreba spremljenog modela	20
13. Primjena modela na slike s interneta i ograničenja	21
14. Zaključak	24

1. Uvod

Automatska analiza parkirnih prostora predstavlja važan problem u području računalnog vida i inteligentnih prometnih sustava. Sustavi koji omogućuju detekciju slobodnih i zauzetih parkirnih mjesta mogu značajno doprinijeti smanjenju prometnih gužvi, emisija štetnih plinova i poboljšanju korisničkog iskustva vozača.

Iako je izvorni cilj projekta bio provesti piksel-razinsku segmentaciju parkirnih zona, korišteni skup podataka ne sadrži segmentacijske maske. Zbog toga je projekt realiziran korištenjem detekcije objekata temeljene na dubokom učenju, što predstavlja praktičnu i često korištenu alternativu segmentaciji u stvarnim sustavima pametnog parkiranja.

U ovom radu koristi se YOLOv8 neuronska mreža za detekciju parkirnih mjesta i njihovu klasifikaciju na slobodna i zauzeta.



Slika 1 Primjer ulazne slike parkirišta iz skupa podataka

2. Računalno okruženje i odabir hardvera

Eksperimenti su provedeni u okruženju **Google Colab**, koje omogućuje izvođenje računalno zahtjevnih zadataka u oblaku. Budući da treniranje dubokih neuronskih mreža uključuje velik broj paralelnih matematičkih operacija (matrična množenja i konvolucije), izvođenje na procesoru (CPU) nije praktično zbog dugog vremena izvođenja.

Zbog toga je korišten **grafički procesor (GPU)**, konkretno **NVIDIA Tesla T4**, uz podršku CUDA tehnologije. CUDA omogućuje izvođenje operacija neuronske mreže na GPU-u, čime se treniranje višestruko ubrzava u odnosu na CPU.

Change runtime type

Runtime type

Python 3 ▼

Hardware accelerator ?

☐ CPU ☒ T4 GPU ☐ H100 GPU ☐ A100 GPU

☐ L4 GPU ☐ v5e-1 TPU ☐ v6e-1 TPU

i Want access to premium GPUs? [Purchase additional compute units](#)

Runtime version ?

Latest (recommended) ▼

Cancel Save

Slika 2 Google Colab postavke – odabir GPU-a (Tesla T4)

3. Spajanje Google Drive-a

Google Colab koristi privremeni datotečni sustav u kojem se podaci brišu nakon završetka sesije. Kako bi se osigurala trajna pohrana skupa podataka, treniranih modela i rezultata, Google Drive je spojen s Colab okruženjem pomoću sljedeće naredbe:

```
from google.colab import drive  
  
drive.mount('/content/drive')
```

Na taj način omogućeno je čitanje i spremanje podataka izravno na Google Drive.

```
from google.colab import drive  
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Slika 3 Uspješno spajanje Google Drive-a u Colabu

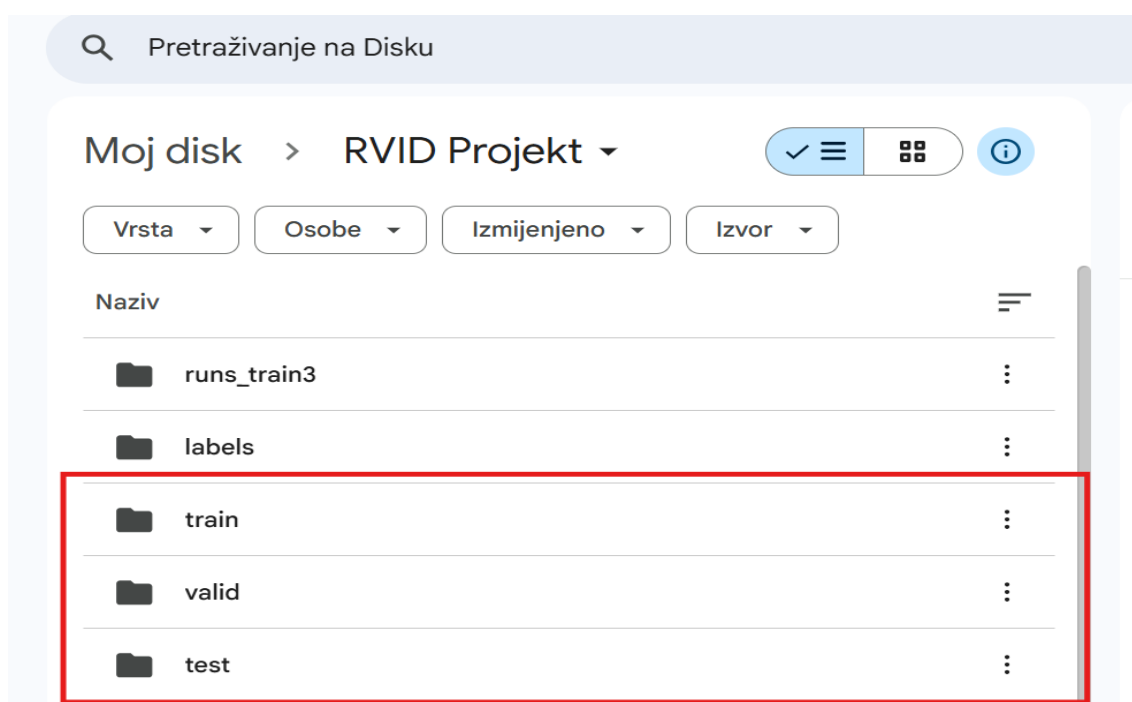
4. Skup podataka i podjela

Skup podataka sastoji se od slika parkirališta snimljenih fiksni nadzornim kamerama u različitim uvjetima osvjetljenja i razinama popunjenosti. U ovom radu korišten je PKLot skup podataka, koji je često korišten referentni dataset za zadatke detekcije i klasifikacije parkirnih mjesta. Dataset je unaprijed podijeljen na tri disjunktna dijela kako bi se omogućilo pravilno treniranje i evaluacija modela:

- Train skup – koristi se za učenje parametara neuronske mreže i sadrži 8 694 slike, od kojih 8 691 slika ima pripadajuće anotacije parkirnih mjesta.

- Validation skup – koristi se tijekom treniranja za praćenje performansi modela i odabir najboljeg modela (best.pt). Ovaj skup sadrži 2 483 slike, pri čemu sve slike imaju odgovarajuće anotacije.
- Test skup – koristi se isključivo za konačnu evaluaciju istreniranog modela i nije uključen u proces učenja. Testni skup sadrži 1 241 sliku, od kojih sve imaju pripadajuće anotacije.

Ovakva podjela osigurava objektivnu procjenu sposobnosti modela da generalizira na nove, prethodno neviđene podatke.



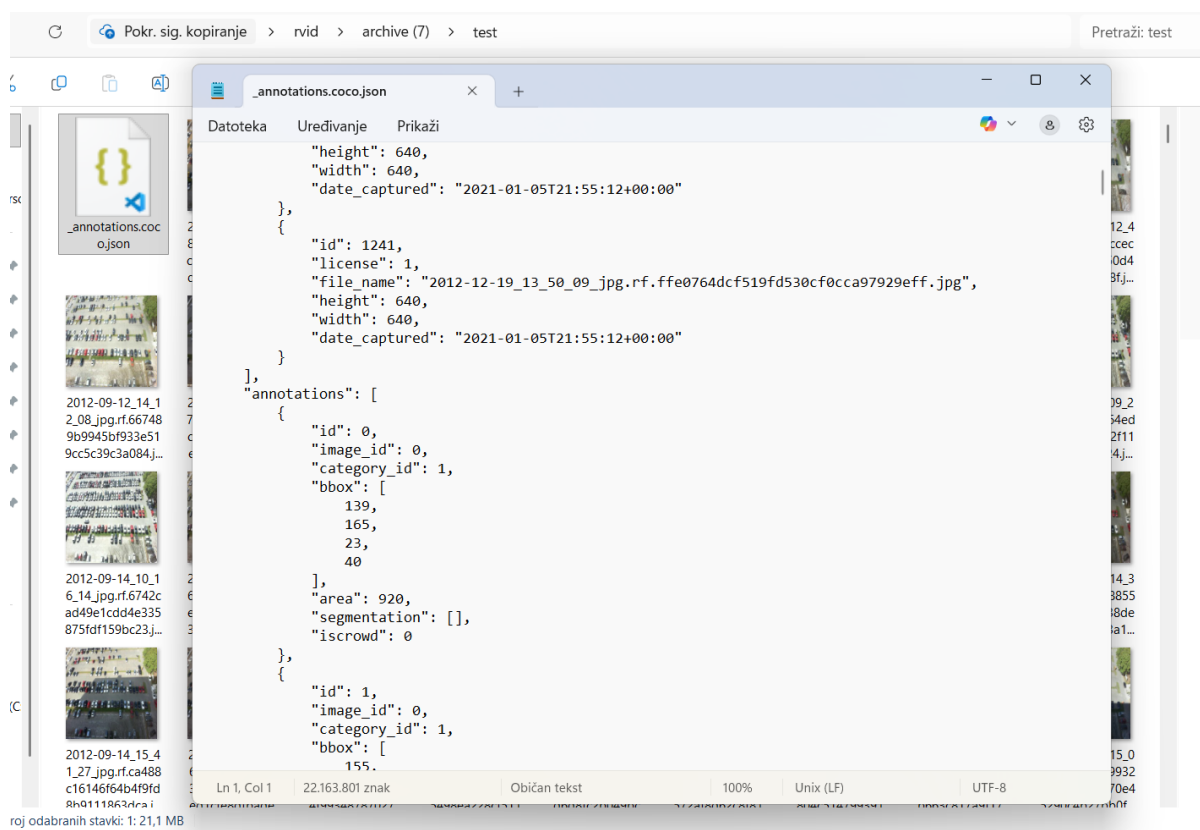
Slika 4 Struktura skupa podataka (train / valid / test)

Podjela ima sljedeću ulogu:

- **train** skup koristi se za učenje težina neuronske mreže,
- **validation** skup koristi se nakon svake epohe treniranja za procjenu performansi i odabir modela best.pt,
- **test** skup koristi se isključivo za konačnu evaluaciju i nije korišten tijekom treniranja.

5. Priprema anotacija: COCO → YOLO format

Izvorne anotacije bile su dostupne u **COCO JSON formatu**, gdje su sve anotacije pohranjene u jednoj JSON datoteci. Budući da YOLOv8 koristi vlastiti format anotacija, bilo je potrebno provesti konverziju.



Slika 5 Primjer COCO JSON anotacije

YOLO format zahtijeva da svaka slika ima zasebnu .txt datoteku s anotacijama u obliku: <class_id> <x_center> <y_center> <width> <height>

Sve vrijednosti su normalizirane u rasponu [0, 1]. Ove vrijednosti predstavljaju numeričke vektore tj. **tenzore** koji omogućuju neuronskoj mreži učenje lokalizacije i klasifikacije objekata.

Za konverziju je implementirana Python skripta koja:

- učitava COCO JSON datoteku,

- mapira kategorije na YOLO indekse klasa,
- za svaku sliku izračunava normalizirane koordinate bounding boxeva,
- generira .txt datoteku za svaku sliku.

```

import json
from pathlib import Path
from PIL import Image

BASE = Path("/content/drive/MyDrive/RVID Projekt")

splits = {
    "train": BASE / "train" / "_annotations.coco.json",
    "valid": BASE / "valid" / "_annotations.coco.json",
    "test": BASE / "test" / "_annotations.coco.json",
}

def coco_to_yolo_labels(coco_json: Path, images_dir: Path, out_labels_dir: Path):
    out_labels_dir.mkdir(parents=True, exist_ok=True)
    data = json.loads(coco_json.read_text())

    # classes
    cats = sorted(data.get("categories", []), key=lambda c: c["id"])
    cat_id_to_idx = {c["id"]: i for i, c in enumerate(cats)}
    class_names = [c["name"] for c in cats]

    # images
    img_map = {}
    for im in data.get("images", []):
        img_map[im["id"]] = (im["file_name"], im.get("width"), im.get("height"))

    # group anns
    ann_by_img = {}
    for ann in data.get("annotations", []):
        if ann.get("iscrowd", 0) == 1:
            continue
        ann_by_img.setdefault(ann["image_id"], []).append(ann)

    written, missing_imgs, total_boxes = 0, 0, 0

    for img_id, (fname, w, h) in img_map.items():
        src = images_dir / fname
        if not src.exists():
            src = images_dir / Path(fname).name
            if not src.exists():
                missing_imgs += 1

        if not w or not h:
            with Image.open(src) as im:
                w, h = im.size

        label_path = out_labels_dir / (Path(src).stem + ".txt")
        lines = []

        for ann in ann_by_img.get(img_id, []):
            cls = cat_id_to_idx.get(ann["category_id"], None)
            if cls is None:
                continue
            x, y, bw, bh = ann["bbox"]

            xc = (x + bw / 2) / w
            yc = (y + bh / 2) / h
            nw = bw / w
            nh = bh / h

            if nw <= 0 or nh <= 0:
                continue

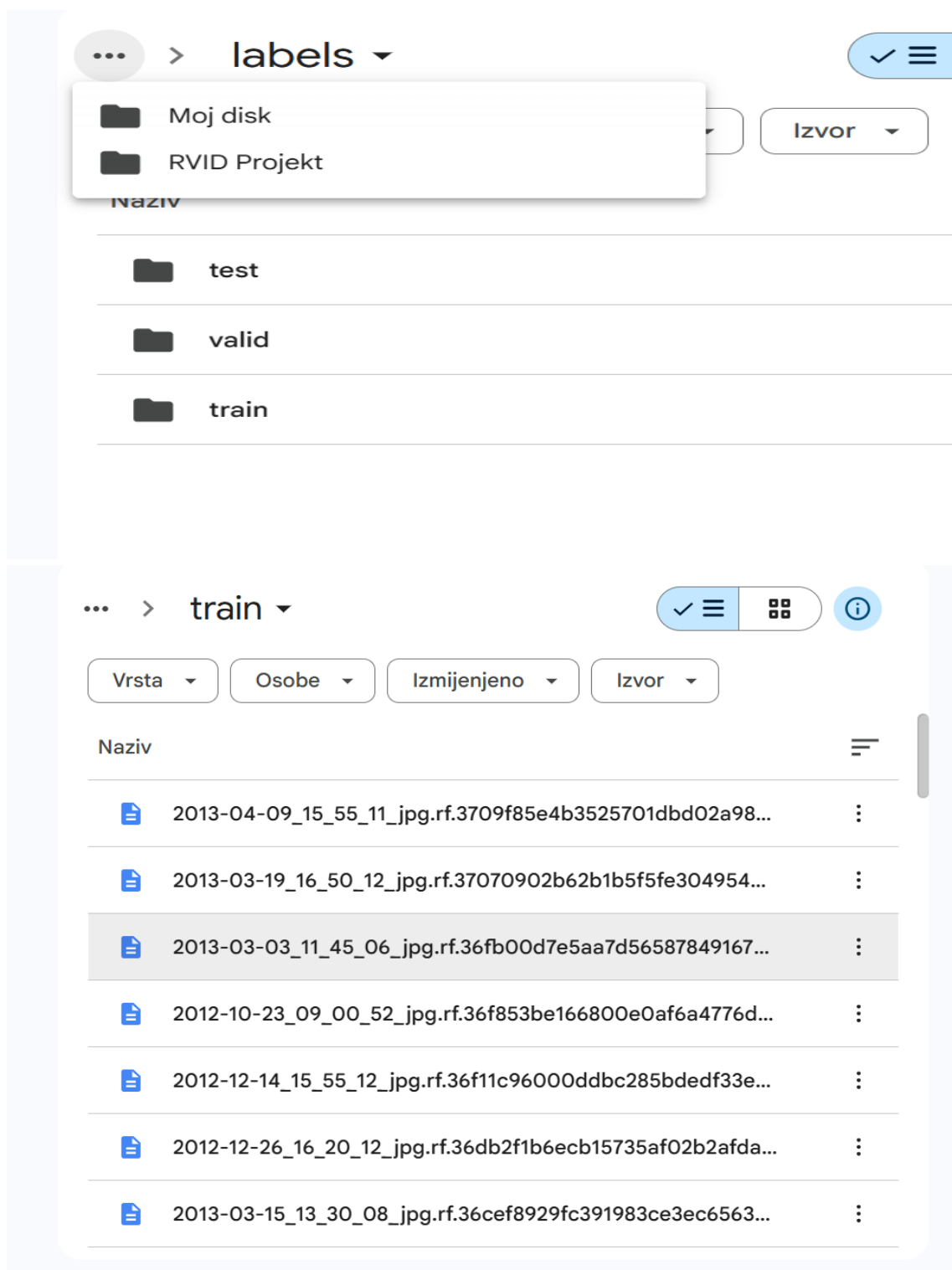
            lines.append(f"{cls} {xc:.6f} {yc:.6f} {nw:.6f} {nh:.6f}")
            total_boxes += 1

        label_path.write_text("\n".join(lines))
        written += 1

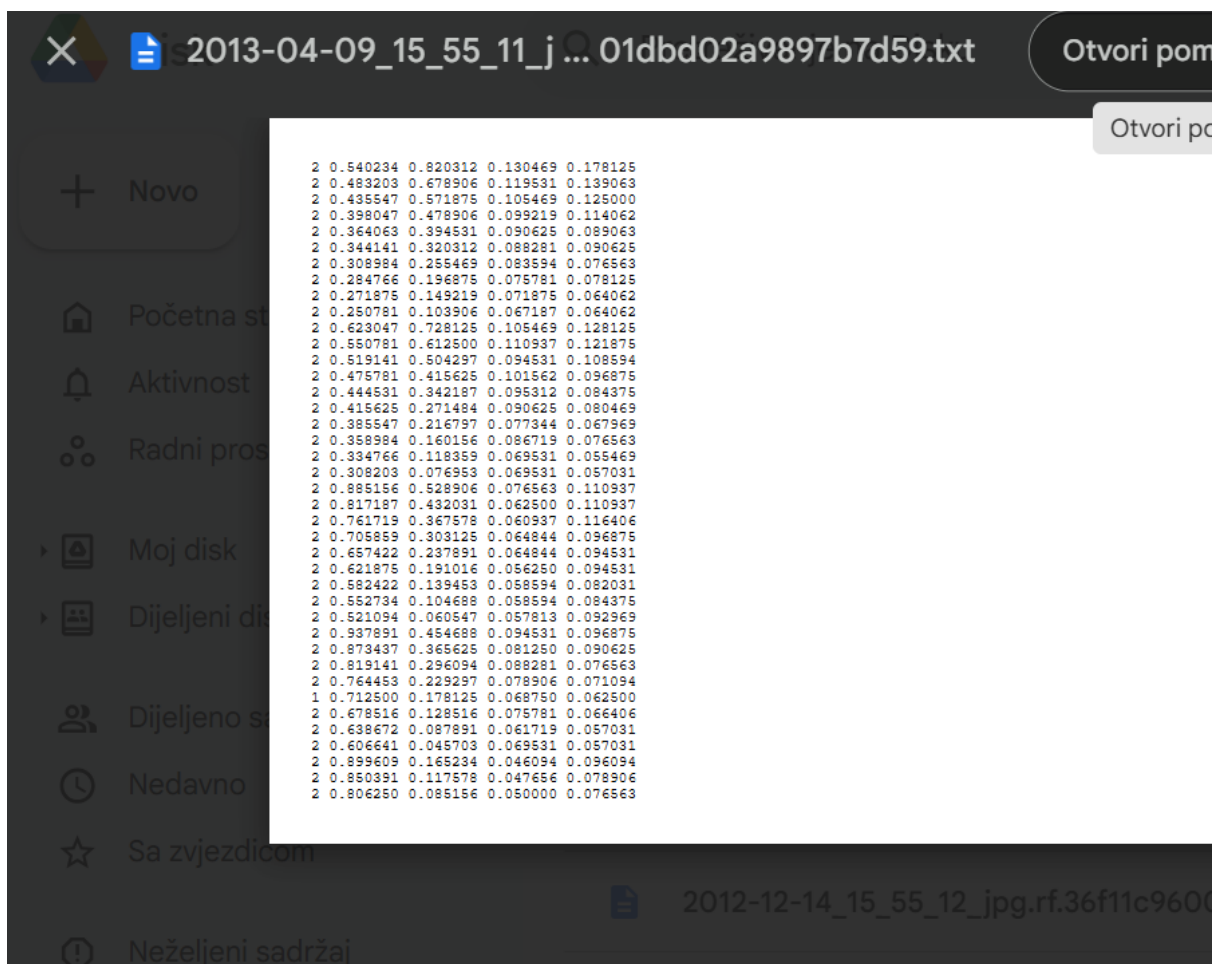
    return class_names, written, missing_imgs, total_boxes

```

Slika 6 Python skripta
za konverziju coco to
yolo



Slika 7 Generirani labels za svaki split



Slika 8 Primjer yolo anotacije

6. Konfiguracijska datoteka data.yaml

Generira se datoteka **data.yaml** koja definira osnovne informacije o datasetu:

```
from pathlib import Path

yaml_path = Path("/content/ds/data.yaml")
yaml_text = """path: /content/ds
train: images/train
val: images/val
test: images/test

names:
  0: spaces
  1: space-empty
  2: space-occupied
"""

yaml_path.write_text(yaml_text)
print("✅ data.yaml:", yaml_path)
print(yaml_text)
```

```
✅ data.yaml: /content/ds/data.yaml
path: /content/ds
train: images/train
val: images/val
test: images/test

names:
  0: spaces
  1: space-empty
  2: space-occupied
```

Slika 9Data.yaml

Ova datoteka YOLO modelu govori:

- gdje se nalaze skupovi podataka,
- koliko klasa postoji,
- koja je semantika pojedine klase.

7. Instalacija YOLOv8 biblioteke

YOLOv8 implementiran je u okviru **Ultralytics** Python biblioteke. Budući da nije unaprijed instalirana u Colabu, instalacija je provedena pomoću:



Slika 10 Instalacija ultralytics biblioteke

Ova biblioteka omogućuje treniranje, evaluaciju i inferenciju YOLOv8 modela.

8. Treniranje YOLOv8 modela

Za treniranje je korišten **YOLOv8n** (nano) model, pretreniran na COCO skupu podataka. Primijenjen je pristup **transfer learninga**, gdje se postojeće težine dodatno prilagođavaju novom problemu.

Osnovni parametri treniranja:

- broj epoha: 30
- veličina ulazne slike: 640 × 640
- batch size: 16
- uređaj: GPU (Tesla T4)

Tijekom treniranja, nakon svake epohe provedena je evaluacija na validation skupu. Model s najboljim performansama automatski je spremljen kao `best.pt`.

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data="/content/ds/data.yaml",
    epochs=30,
    imgsz=640,
    batch=16,
    device=0
)

20          [-1, 9] 1 0 ultralytics.nn.modules.conv.Concat [1]
21          [-1, 1] 1 493056 ultralytics.nn.modules.block.C2f [384, 256, 1]
22          [15, 18, 21] 1 751897 ultralytics.nn.modules.head.Detect [3, 16, None, [64, 128, 256]]
Model summary: 130 layers, 3,011,433 parameters, 3,011,417 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed ✓
train: Fast image access ✓ (ping: 0.3±0.1 ms, read: 27.0±6.2 MB/s, size: 60.7 KB)
train: Scanning /content/ds/labels/train... 8691 images, 192 backgrounds, 0 corrupt: 100% ————— 8694/8694 1
train: New cache created: /content/ds/labels/train.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='wei
val: Fast image access ✓ (ping: 0.4±0.2 ms, read: 13.9±5.1 MB/s, size: 66.2 KB)
val: Scanning /content/ds/labels/val... 2483 images, 59 backgrounds, 0 corrupt: 100% ————— 2483/2483 170.8i
val: New cache created: /content/ds/labels/val.cache
Plotting labels to /content/runs/detect/train3/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0
optimizer: AdamW(lr=0.001429, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 6
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/detect/train3
Starting training for 30 epochs...
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/30	6.05G	1.46	1.281	1.113	540	640: 100% ————— 544/544 2.8it/s 3
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% ————— 78/78
	all	2483	143316	0.94	0.961	0.969 0.701

Slika 11Ispis pokretanja treniranja YOLOv8 modela

results.csv

Otvori pomoću aplikacije

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
epoch	time	train/box_loss	train/cls_loss	train/dfl_loss	metric/precision(B)	metric/recall(B)	metric/mAP50(B)	metric/mAP50-95(B)	val/box_loss	val/cls_loss	val/dfl_loss	lr0p0	lr0p1	lr0p2	
1	222.284	1.45973	1.28132	1.11252	0.94007	0.9608	0.96878	0.70094	1.01064	46.5074	0.93978	0.000475458	0.000475458	0.000475458	
2	435.959	0.99993	0.62951	0.92752	0.95902	0.96978	0.97743	0.77149	0.80958	inf	0.87597	0.000920382	0.000920382	0.000920382	
3	648.347	0.87327	0.53812	0.89207	0.95724	0.9705	0.9897	0.8211	0.87239	51.2075	0.83904	0.00133387	0.00133387	0.00133387	
4	855.743	0.80197	0.49352	0.87601	0.97384	0.97553	0.99168	0.83861	0.82996	15.8934	0.82996	0.00128753	0.00128753	0.00128753	
5	1064.06	0.7403	0.46377	0.86108	0.96034	0.97112	0.99186	0.85257	0.60134	14.4017	0.82149	0.00124037	0.00124037	0.00124037	
6	1273.88	0.69903	0.44282	0.85227	0.96304	0.97214	0.99236	0.87303	0.53333	8.81473	0.80701	0.00119322	0.00119322	0.00119322	
7	1484.8	0.65945	0.41286	0.84478	0.94706	0.96485	0.99204	0.88488	0.50405	3.63389	0.79921	0.00114606	0.00114606	0.00114606	
8	1693.26	0.64506	0.40312	0.84105	0.96839	0.9875	0.99376	0.87445	0.54815	3.40866	0.81139	0.0010989	0.0010989	0.0010989	
9	1901.25	0.61686	0.39071	0.83641	0.98424	0.98468	0.99267	0.8944	0.46777	4.83784	0.79091	0.00105174	0.00105174	0.00105174	
10	2108.7	0.5945	0.37994	0.83159	0.99152	0.99126	0.99436	0.91365	0.4299	0.43323	0.78364	0.00100459	0.00100459	0.00100459	
11	2315.13	0.58057	0.36335	0.83025	0.98756	0.98136	0.99296	0.91119	0.42433	22.887	0.78295	0.00095743	0.00095743	0.00095743	
12	2522.76	0.56757	0.3552	0.82759	0.98972	0.98881	0.99392	0.9192	0.40098	3.72114	0.77778	0.000910273	0.000910273	0.000910273	
13	2733.19	0.55418	0.34779	0.82504	0.99386	0.99432	0.9941	0.9226	0.40617	2.41538	0.77781	0.000863116	0.000863116	0.000863116	
14	2941.3	0.5454	0.33881	0.82346	0.99543	0.99462	0.9943	0.92832	0.38631	2.40832	0.77752	0.000819599	0.000819599	0.000819599	
15	3147.96	0.52355	0.32491	0.82005	0.99453	0.99475	0.99418	0.92794	0.38726	1.47919	0.77482	0.000768802	0.000768802	0.000768802	
16	3354.57	0.51332	0.32098	0.81813	0.99072	0.9923	0.99402	0.93698	0.3621	1.65769	0.77066	0.000721645	0.000721645	0.000721645	
17	3564.46	0.50607	0.31697	0.81677	0.99589	0.99607	0.99433	0.94211	0.34718	2.75887	0.76883	0.000674488	0.000674488	0.000674488	
18	3772.14	0.50014	0.31115	0.81576	0.99635	0.99688	0.99444	0.9442	0.3499	3.90769	0.76907	0.000627331	0.000627331	0.000627331	
19	3980.53	0.49015	0.30487	0.81439	0.99719	0.99697	0.99438	0.94567	0.34488	0.71185	0.76791	0.000580174	0.000580174	0.000580174	
20	4189.13	0.47691	0.29702	0.81259	0.99701	0.99644	0.99441	0.94888	0.33631	1.82814	0.76538	0.000533017	0.000533017	0.000533017	
21	4387.9	0.42152	0.26688	0.80145	0.99768	0.99836	0.99455	0.95151	0.33879	0.57259	0.76676	0.00048586	0.00048586	0.00048586	
22	4581.7	0.39972	0.25206	0.79681	0.99697	0.99725	0.99453	0.95453	0.32241	0.25888	0.76402	0.000438703	0.000438703	0.000438703	
23	4776.81	0.38585	0.24627	0.79477	0.99783	0.99782	0.99458	0.96105	0.2987	0.25999	0.76012	0.000391546	0.000391546	0.000391546	
24	4972.63	0.37295	0.23804	0.79238	0.99846	0.9981	0.99453	0.96387	0.294	0.44374	0.76007	0.000344389	0.000344389	0.000344389	
25	5164.01	0.36017	0.23223	0.79035	0.99811	0.99845	0.99454	0.96648	0.27695	0.28239	0.75887	0.000297232	0.000297232	0.000297232	
26	5359.68	0.35022	0.22742	0.7886	0.99777	0.9983	0.99455	0.96849	0.27515	0.23438	0.75807	0.000250075	0.000250075	0.000250075	
27	5554.27	0.33986	0.22167	0.78746	0.99821	0.99836	0.99454	0.96989	0.26895	0.67288	0.75574	0.000202918	0.000202918	0.000202918	
28	5749.74	0.33265	0.21899	0.78597	0.99825	0.99843	0.99458	0.97204	0.26045	0.28085	0.75404	0.000155761	0.000155761	0.000155761	
29	5941.19	0.32414	0.21517	0.7847	0.99831	0.99848	0.99454	0.9736	0.25782	0.19997	0.75412	0.000108604	0.000108604	0.000108604	
30	6136.42	0.31565	0.21118	0.78343	0.99831	0.99856	0.99458	0.97373	0.25775	0.21791	0.75399	6.14E-05	6.14E-05	6.14E-05	

Slika 12Rezultati treniranja za svaku epohu - results.png

9. Evaluacija modela

9.1 Evaluacija na validation skupu

Validation skup korišten je tijekom treniranja za praćenje performansi i odabir najboljeg modela.

9.2 Konačna evaluacija na testnom skupu

Nakon treniranja, model `best.pt` evaluiran je na testnom skupu pomoću:

```
from ultralytics import YOLO

model = YOLO("/content/runs/detect/train3/weights/best.pt")
model.val(data="/content/ds/data.yaml", split="test")
```

Slika 13 Evaluacija modela na testnom skupu

Dobivene su sljedeće evaluacijske metrike:

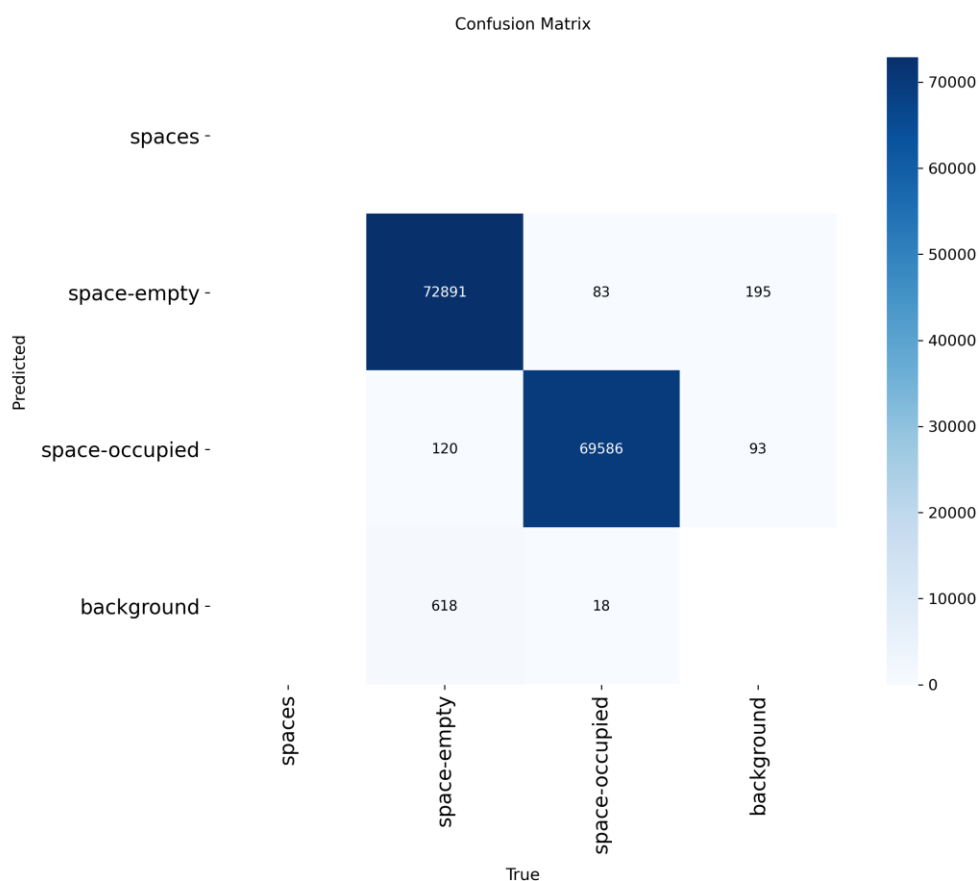
- Precision (0.997) predstavlja omjer ispravno detektiranih parkirnih mjesta u odnosu na ukupan broj detekcija koje je model označio kao pozitivne. Visoka vrijednost precisiona znači da model vrlo rijetko pogrešno označava slobodno ili zauzeto parkirno mjesto kada ono to zapravo nije.
- Recall (0.997) označava omjer ispravno detektiranih parkirnih mjesta u odnosu na ukupan broj stvarnih parkirnih mjesta prisutnih na slici. Visoka vrijednost recalla ukazuje na to da model uspješno pronalazi gotovo sva parkirna mjesta i rijetko propušta postojeće objekte.
- mAP@0.50 (0.994) predstavlja srednju prosječnu preciznost pri pragu preklapanja (IoU) od 0.50. Ova metrika mjeri koliko su predviđeni bounding boxevi dobro prostorno poravnati sa stvarnim anotacijama te ukazuje na vrlo preciznu lokalizaciju parkirnih mjesta.
- mAP@0.50–0.95 (0.977) je stroža metrika koja računa prosječnu preciznost za više IoU pragova (od 0.50 do 0.95). Visoka vrijednost ove metrike pokazuje da model zadržava visoku točnost detekcije i pri zahtjevnijim uvjetima prostorne preciznosti, što ukazuje na dobru generalizaciju modela.

Ovi rezultati potvrđuju da trenirani model postiže vrlo visoku točnost u detekciji i klasifikaciji parkirnih mjesta te je prikladan za primjenu u stvarnim sustavima pametnog parkiranja.

IoU (Intersection over Union) je mjera preklapanja između predviđenog bounding boxa i stvarne (ground truth) anotacije. Definira se kao omjer površine presjeka i površine unije ta dva pravokutnika. Vrijednost IoU kreće se u rasponu od 0 do 1, pri čemu veće vrijednosti označavaju bolje prostorno poklapanje predikcije s anotacijom.

U ovom radu IoU se koristi za procjenu kvalitete lokalizacije parkirnih mjesta, pri čemu se mAP@0.50 smatra ispravnom detekcijom ako je IoU veći od 0.50, dok mAP@0.50–0.95 predstavlja strožu evaluaciju kroz više pragova preklapanja.

9.3 Analiza konfuzijske matrice



Slika 14 Konfuzijska matrica

Na slici je prikazana **konfuzijska matrica** dobivena evaluacijom modela na testnom skupu podataka. Konfuzijska matrica prikazuje odnos između **stvarnih klasa (True)** i **predviđenih klasa (Predicted)** te omogućuje detaljan uvid u vrste pogrešaka koje model čini.

Točne klasifikacije

Velike vrijednosti na glavnoj dijagonali matrice ukazuju na visoku točnost modela:

- **72 891** slobodno parkirno mjesto (space-empty) ispravno je klasificirano kao slobodno.
- **69 586** zauzetih parkirnih mjesta (space-occupied) ispravno je klasificirano kao zauzeto.

Ovi rezultati potvrđuju da model vrlo pouzdano razlikuje slobodna i zauzeta parkirna mjesta.

Pogrešne klasifikacije

Manji broj pogrešaka prisutan je izvan glavne dijagonale:

- **83** slobodna mjesta pogrešno su klasificirana kao zauzeta.
- **120** zauzetih mjesta pogrešno je klasificirano kao slobodno.

Ove pogreške su rijetke i mogu biti posljedica djelomično zaklonjenih vozila, sjenki ili slabijih uvjeta osvjetljenja.

Klasa **background** ne predstavlja stvarnu kategoriju objekta iz skupa podataka, već označava:

- područja slike gdje **nema parkirnog mjesta**,
- slučajeve kada model predvidi ili propusti detekciju objekta.

Vrijednosti vezane uz background (npr. 618 i 18) odnose se na situacije u kojima je model:

- propustio detektirati postojeće parkirno mjesto (false negative),

- ili pogrešno detektirao objekt tamo gdje on ne postoji (false positive).

Mali broj takvih slučajeva ukazuje na dobru sposobnost modela da razlikuje relevantne objekte od pozadine.

10. Vizualna demonstracija (inferencija)

Za kvalitativnu analizu provedena je inferencija modela na testnim slikama pomoću:

```
model.predict(source="/content/ds/images/test", conf=0.25, save=True)

image 6/1241 /content/ds/images/test/2012-09-12_11_19_23_jpg.rf.6117980a959
image 7/1241 /content/ds/images/test/2012-09-12_11_56_00_jpg.rf.041680c3e77
```

Rezultati su spremljeni kao slike s označenim parkirnim mjestima, gdje su slobodna i zauzeta mjesta vizualno razlikovana.



Slika 15 Primjer detekcije slobodnih i zauzetih parkirnih mjesta

11. Spremanje rezultata

Zbog privremene prirode Colab okruženja, svi rezultati treniranja i evaluacije kopirani su na Google Drive, uključujući:

- best.pt i last.pt,
- grafove učenja,
- confusion matrix,
- primjere detekcija.

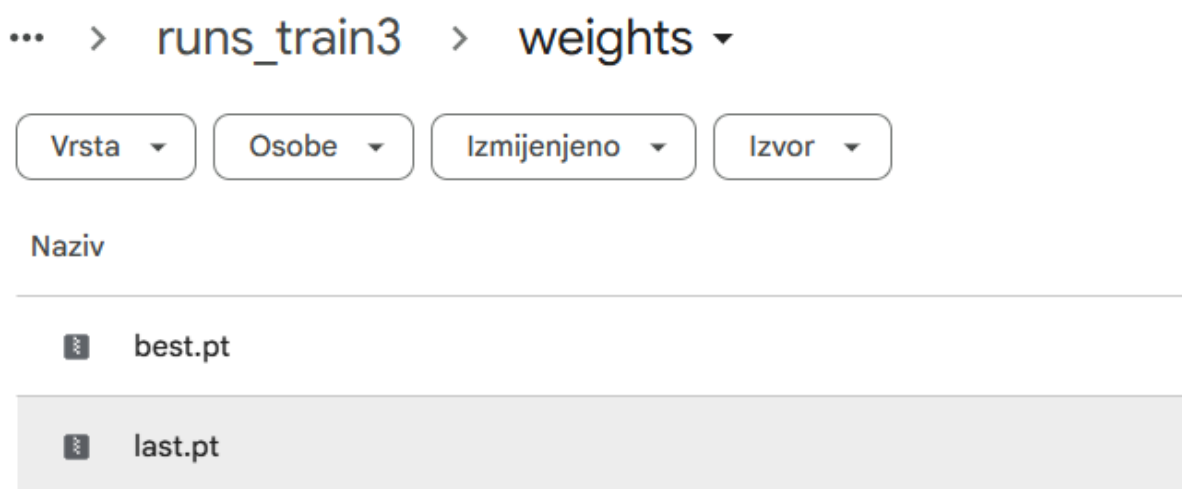
Na taj način osigurana je trajna pohrana rezultata.

```
from pathlib import Path src = Path("/content/runs/detect/train3")
dst = Path("/content/drive/MyDrive/RVID Projekt/runs_train3")

if dst.exists():
    shutil.rmtree(dst)

shutil.copytree(src, dst)
```

Slika 16 Spremanje rezultata na disk



Slika 17 Spremljeni modeli

Moj disk > RVID Projekt > runs_train3 ▾

Vrsta ▾ Osobe ▾ Izmijenjeno ▾ Izvor ▾

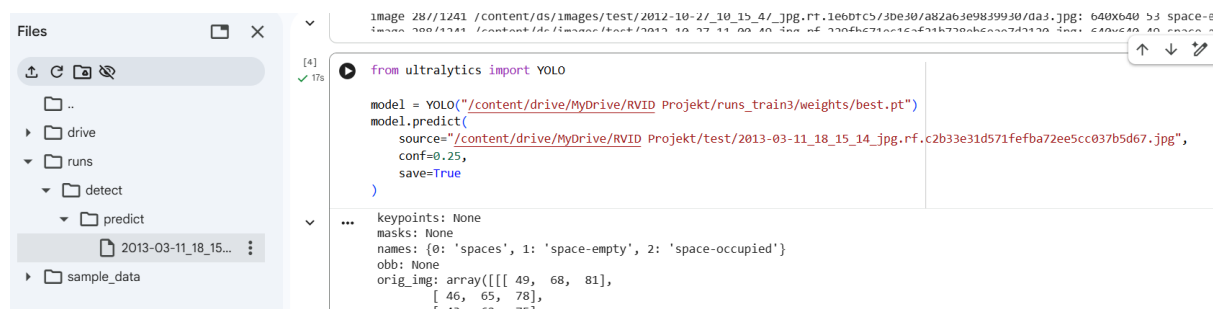
Naziv	Vlasnik	Datum izmjene ↓
 weights	 ja	8. velj ja
 results.png	 ja	8. velj ja
 confusion_matrix.png	 ja	8. velj ja
 confusion_matrix_normalized.png	 ja	8. velj ja
 BoxR_curve.png	 ja	8. velj ja
 BoxP_curve.png	 ja	8. velj ja
 BoxF1_curve.png	 ja	8. velj ja
 BoxPR_curve.png	 ja	8. velj ja
 val_batch2_labels.jpg	 ja	8. velj ja
 val_batch2_pred.jpg	 ja	8. velj ja
 val_batch1_labels.jpg	 ja	8. velj ja
 val_batch1_pred.jpg	 ja	8. velj ja
 val_batch0_pred.jpg	 ja	8. velj ja
 val_batch0_labels.jpg	 ja	8. velj ja
 results.csv	 ja	8. velj ja

Slika 18Spremljeni rezultati i modeli

12. Ponovna upotreba spremljenog modela

Nakon završetka treniranja i evaluacije modela, najbolji model (best.pt) trajno je spremljen na Google Drive kako bi se omogućila njegova ponovna upotreba bez potrebe za ponovnim treniranjem. Ovakav pristup je uobičajen u praksi te omogućuje jednostavnu primjenu istreniranog modela na nove, prethodno neviđene podatke.

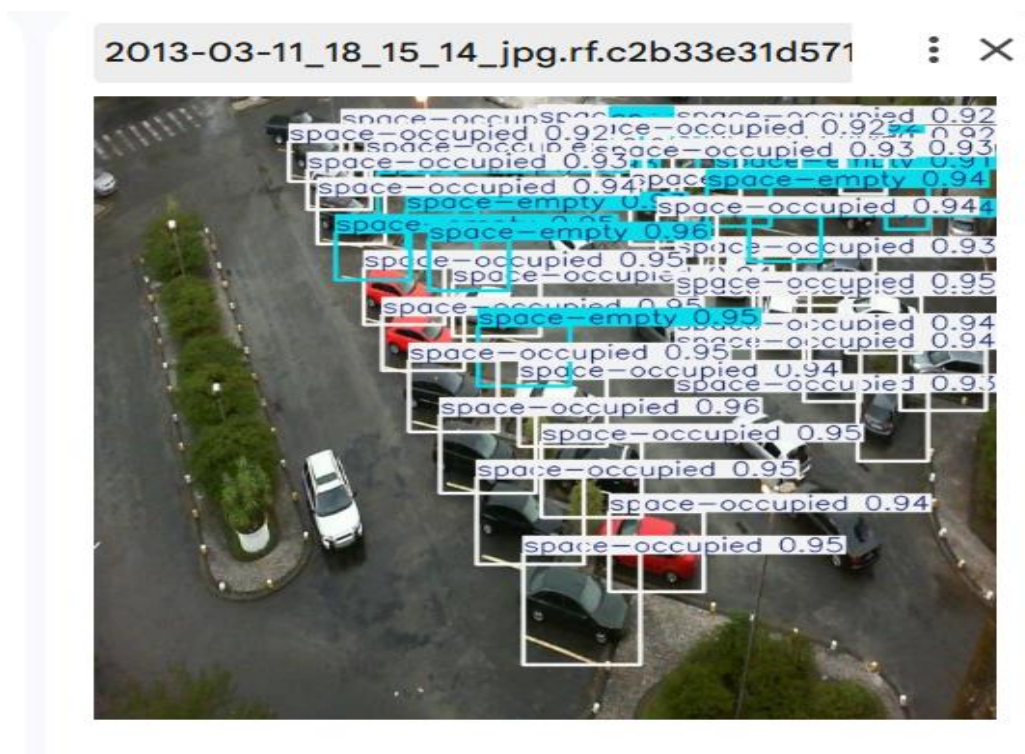
U ovom koraku demonstrirana je predikcija modela na jednoj slici parkirališta iz testnog skupa. Model se učitava iz spremljene lokacije na disku te se koristi za inferenciju pomoću YOLOv8 biblioteke. Time se simulira stvarna primjena sustava u kojoj se model koristi za analizu pojedinačnih slika ili videozapisa.



Slika 19 Uporaba spremljenog modela

Tijekom izvođenja predikcije, model analizira ulaznu sliku te detektira parkirna mjesta, pri čemu ih klasificira kao slobodna ili zauzeta. Parametar conf definira minimalnu razinu pouzdanosti detekcije, dok opcija save=True omogućuje spremanje rezultata u obliku slike s ucrtanim graničnim okvirima i oznakama klasa.

Rezultat predikcije pohranjuje se u direktorij runs/detect/predict, gdje je moguće vizualno analizirati uspješnost detekcije. Ovaj primjer potvrđuje da se istrenirani model može učinkovito koristiti za automatsku analizu novih slika bez dodatnog učenja, što je ključno za primjenu u stvarnim sustavima pametnog parkiranja.



Slika 20 Primjer predikcije spremljenog YOLOv8 modela na jednoj slici parkirišča

13. Primjena modela na slike s interneta i ograničenja

Nakon treniranja i evaluacije modela, provedeno je dodatno testiranje istreniranog YOLOv8 modela na pojedinačnim slikama parkirišča preuzetima s interneta. Cilj ovog testiranja bio je provjeriti sposobnost modela da generalizira na nove, prethodno neviđene podatke koji se razlikuju od onih korištenih u skupu za učenje.


```

from ultralytics import YOLO

model = YOLO("/content/drive/MyDrive/RVID Projekt/runs_train3/weights/best.pt")
model.predict(
    source="/content/drive/MyDrive/RVID Projekt/slike parkinga s interneta/parking_4.jpg",
    conf=0.20,
    save=True
)

...
image 1/1 /content/drive/MyDrive/RVID Projekt/slike parkinga s interneta/parking_4.jpg: 448x640 21 spa
Speed: 2.5ms preprocess, 7.1ms inference, 1.2ms postprocess per image at shape (1, 3, 448, 640)
Results saved to /content/runs/detect/predict6
[ultralytics.engine.results.Results object with attributes:

boxes: ultralytics.engine.results.Boxes object
keypoints: None
masks: None
names: {0: 'spaces', 1: 'space-empty', 2: 'space-occupied'}
obb: None
orig_img: array([[ 55,  53,  53],
 [ 57,  55,  55],
 [ 58,  56,  56],
 ...,
 [ 54,  52,  52],
 [ 53,  51,  51],

```

Slika 21 Predikcija na slici s interneta

Model je pokazao sposobnost detekcije i klasifikacije parkirnih mjesta i na slikama koje nisu bile dio izvornog skupa podataka. Međutim uočene su određene nepravilnosti u rezultatima. Na nekim slikama detekcije su bile manje precizne, a pojedina parkirna mjesta nisu bila ispravno klasificirana ili uopće detektirana.



Slika 22 Detekcija parking mjesta parking_4.jpg



Slika 23 Detekcija parking mjesta parking1.jpg

Glavni razlog smanjene točnosti na slikama s interneta leži u razlici distribucije podataka u odnosu na izvorni skup podataka (PKLot). Slike s interneta često su snimljene iz drugačijih kutova, s viših ili nižih perspektiva, pri različitim rezolucijama te u uvjetima osvjetljenja koji nisu bili dovoljno zastupljeni u trening skupu. Također, raspored parkirnih mjesta, oznake na tlu i tipovi vozila mogu značajno odstupati od onih viđenih tijekom treniranja.

Dodatno ograničenje predstavlja činjenica da je model treniran isključivo na statičnim slikama fiksnih nadzornih kamera, dok slike s interneta često dolaze iz slobodnih izvora, mobilnih uređaja ili dronova, što dodatno otežava generalizaciju.

Unatoč navedenim ograničenjima, rezultati pokazuju da model i dalje zadržava osnovnu sposobnost detekcije parkirnih mjesta, što potvrđuje ispravnost odabranog pristupa. Daljnje poboljšanje performansi bilo bi moguće proširenjem skupa podataka slikama s različitih lokacija i perspektiva te dodatnim treniranjem modela.

14. Zaključak

U ovom projektu razvijen je sustav za automatsku detekciju parkirnih mjesta i njihovu klasifikaciju na slobodna i zauzeta korištenjem metoda dubokog učenja. Primjenom YOLOv8 detekcijskog modela ostvarena je visoka razina točnosti i pouzdanosti, što potvrđuju dobivene evaluacijske metrike na testnom skupu podataka. Rezultati pokazuju da model uspješno generalizira na nove, prethodno neviđene slike parkirališta te ispravno prepoznaje stanje parkirnih mjesta u različitim uvjetima osvjetljenja i popunjenosti.

Posebna prednost korištenog pristupa je mogućnost rada u gotovo stvarnom vremenu, što ga čini prikladnim za primjenu u sustavima pametnog parkiranja i inteligentnog upravljanja prometom. Dodatno, sustav omogućuje jednostavnu ponovnu upotrebu istreniranog modela, čime se olakšava njegova integracija u stvarne aplikacije bez potrebe za ponovnim treniranjem.

Razvijeni sustav može poslužiti kao temelj za daljnji razvoj naprednih rješenja u području upravljanja parkiralištima, optimizacije prometnih tokova i poboljšanja korisničkog iskustva vozača. Budući rad može uključivati proširenje sustava na analizu videozapisa u stvarnom vremenu, prilagodbu modela novim lokacijama te integraciju s vanjskim sustavima, poput mobilnih aplikacija ili nadzornih platformi, radi prikaza informacija o dostupnosti parkirnih mjesta.